FILES & EXCEPTIONS

INFO1 Mission 6 (S7)

Jean-Luc Bibaud



Opening a file

```
file = open(STRING_OF_FILE_LOCATION, MODE)
```

The main modes

`"r"`	**read** (the default if you leave it out)
`"w"`	**write**: erases the file first!
`"a"`	**append**: adds at the end of the file

Don't forget to close it...

```
file = open("notes.txt", "r")
...                 # use the file
file.close()
```

...or let with do it for you

```
with open("notes.txt") as file:
    text = file.read()
# closed automatically here
```

Reading a file

```
file = open("notes.txt")

file.read(3)          # NUM_OF_CHARS
→ 'hel'

file.readline()       # reads until newline
→ 'lo\n'    ← continues where it stopped!

file.readline()
→ 'world\n'

file.readline()
→ ''    ← end of the file

file.close()
```

notes.txt

```
hello\n
world\n
```

The file **remembers where you stopped**.

Why end=""?

```
print(file.readline())
→ hello
  (+ an empty line)

print(file.readline(), end="")
→ world
```

Looping over lines in a file, writing to a file

Looping over the lines

```
with open("notes.txt") as file:
        for line in file.readlines():
                print(line, end="")
```

```
→ hello
  world
```

`readlines()` gives a list of all lines. Shorter version that does the same: **`for line in file:`**

Writing to a file

```
with open("out.txt", "w") as file:
        file.write("This is a test\n")
        file.write("Second line\n")
```

out.txt now contains:

```
→ This is a test
  Second line
```

`write()` does **not** add the newline for you: put the **`\n`** yourself. Use **`"a"`** to add to a file instead of erasing it.

ERRORS

Exceptions

```
try:
        n = int(input("Number? "))
        print(10 / n)
except ValueError as error:  # specific one
        print("Not a number:", error)
except ZeroDivisionError:
        print("Can't divide by 0")
```

```
Number? abc
→ Not a number: invalid literal for int()
  with base 10: 'abc'

Number? 0    → Can't divide by 0

Number? 4    → 2.5
```

Catch them with try / except

If a line in try raises an error, Python jumps to the matching except instead of crashing.

Ones you'll meet a lot:

`ValueError`

`TypeError`

`IndexError`

`ZeroDivisionError`

`FileNotFoundError`

A bare **`except:`** catches everything, even your typos. Name the exception you expect.